

Guide CI/CD — La Termitière

Version : 1.0
Statut : Production
Cible : Équipe technique / DevOps
Dernière mise à jour : Juillet 2026

1. Objectif

Ce document décrit le pipeline d'intégration et de déploiement continu (CI/CD) pour le Frontend React (`termitiere-web`).
Le déploiement est 100% automatisé grâce à GitHub Actions et GitHub Container Registry (GHCR).

2. Architecture du Pipeline

Le flux de déploiement fonctionne de la manière suivante :

- Les développeurs collaborent sur la branche `dev` (ou des branches de feature).
- Quand le code est prêt, il est poussé (merge) sur la branche `vps-deploy`.
- GitHub Actions se déclenche automatiquement (uniquement sur cette branche).
- Le code est compilé (Build Vite/React) et packagé dans une image Docker.
- L'image Docker est poussée sur le registre privé de GitHub (GHCR).
- GitHub Actions se connecte au VPS via SSH.
- Le VPS télécharge (pull) la nouvelle image et redémarre le conteneur Frontend via `docker compose`.

3. Prérequis (GitHub Secrets)

Pour que GitHub Actions puisse compiler l'application et se connecter au VPS, il faut configurer les Secrets suivants dans les paramètres du dépôt GitHub (*Settings > Secrets and variables > Actions*) :

Nom du Secret	Description
<code>HOST</code>	L'adresse IP du VPS (ex: <code>31.207.37.96</code>)
<code>USERNAME</code>	L'utilisateur SSH (ex: <code>bawa</code> ou <code>root</code>)
<code>SSH_PRIVATE_KEY</code>	La clé SSH privée (Ed25519) pour se connecter au VPS sans mot de passe. Voir la section 3.1.
<code>CR_PAT</code>	Le Personal Access Token (PAT) GitHub pour que le VPS puisse pull l'image depuis GHCR
<code>VITE_SUPABASE_URL</code>	L'URL publique de l'API (ex: <code>https://api.latermitiere.com</code>)
<code>VITE_SUPABASE_ANON_KEY</code>	La clé publique anonyme générée par Supabase

3.1 Génération de la clé SSH (github-actions)

Pour permettre à GitHub de se connecter au VPS :

- Sur le VPS, on génère une clé asymétrique dédiée (sans mot de passe) :

```
ssh-keygen -t ed25519 -C "github-actions" -f ~/.ssh/github-actions
```

- On autorise cette clé sur le VPS :

```
cat ~/.ssh/github-actions.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

- On copie le contenu de la clé privée (`cat ~/.ssh/github-actions`) et on le colle dans le secret `SSH_PRIVATE_KEY` sur GitHub.

⚠ Piège classique : Assure-toi de copier l'intégralité du texte, y compris les lignes `-----BEGIN OPENSSH PRIVATE KEY-----` et `-----END OPENSSH PRIVATE KEY-----`. S'il manque un tiret, la connexion GitHub Actions échouera.

4. Le Personal Access Token (PAT) — CRITIQUE

Le VPS a besoin d'une autorisation pour télécharger l'image Docker privée depuis GHCR. C'est le rôle du `CR_PAT`.

4.1 Création du PAT

- Sur GitHub, va dans *Settings (Profil) > Developer settings > Personal access tokens > Tokens (classic)*.
- Clique sur *Generate new token (classic)*.
- Nom : `VPS Deploy Token La Termitiere`.
- Expiration : **90 jours** (Standard de sécurité).
- Scopes (Permissions) : Coche `read:packages` (c'est la seule permission requise pour pull l'image).
- Copie le token généré (il commence par `ghp_...`).

4.2 Injection sur le VPS

Sur le VPS, il faut s'authentifier une fois pour toutes auprès du registre.

(Remplace `ghp_TON_TOKEN_PAT` par ton vrai token, et `TON_USER_GITHUB` par ton nom d'utilisateur GitHub en minuscules) :

```
echo "ghp_TON_TOKEN_PAT" | \
  docker login ghcr.io -u TON_USER_GITHUB --password-stdin
```

(Le VPS est maintenant autorisé à télécharger les mises à jour privées).

5. Procédure de Rotation du PAT (Tous les 90 jours)

⚠ **Attention** : Le PAT expire tous les 90 jours. Si le pipeline CI/CD plante subitement avec une erreur `unauthorized` au moment du `docker pull` sur le VPS, c'est que le token a expiré.

En cas d'expiration (Procédure en 5 minutes) :

- Retourne sur GitHub et génère un nouveau PAT (même procédure que 4.1).
- Met à jour le secret `CR_PAT` dans les paramètres Actions du dépôt.
- Connecte-toi au VPS via SSH et relance l'authentification :

```
echo "ghp_NOUVEAU_TOKEN" | \
  docker login ghcr.io -u TON_USER_GITHUB --password-stdin
```

- Relance le job GitHub Actions qui avait échoué.

6. Tolérance aux pannes et Déploiement "Zero Downtime"

Le workflow GitHub Actions (`ghcr-test.yml`) intègre une stratégie de Rollback Automatique extrêmement robuste, indispensable en production :

- Avant de télécharger la nouvelle version, le script sauvegarde l'identifiant (tag) de l'image actuellement en production.
- Il télécharge et déploie la nouvelle image.
- Il effectue un test de santé (Health Check) en vérifiant que le domaine (`https://app.latermitiere.com`) répond correctement avec `curl`.
- En cas d'échec (Erreur 500 ou site indisponible) : Le script restaure immédiatement l'ancien tag (Rollback) et relance l'ancienne version, assurant qu'il n'y ait presque aucune interruption de service pour l'utilisateur final.